

Computer Networks (2025-2026)

Part 4: Network Layer

Jeroen Famaey

Andrei Belogaev

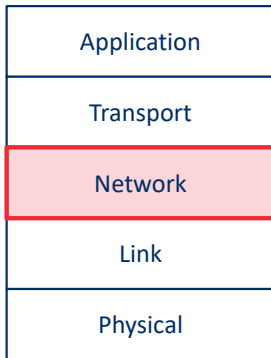
{jeroen.famaey, andrei.belogaev}@uantwerpen.be

Table of contents

1. Network layer overview
2. The Internet Protocol (IPv4 and IPv6)
3. Routing algorithms
4. Routing on the Internet (OSPF and BGP)
5. Software-defined networking (SDN)
6. Network management (ICMP, SNMP, NETCONF, YANG)

Network layer overview

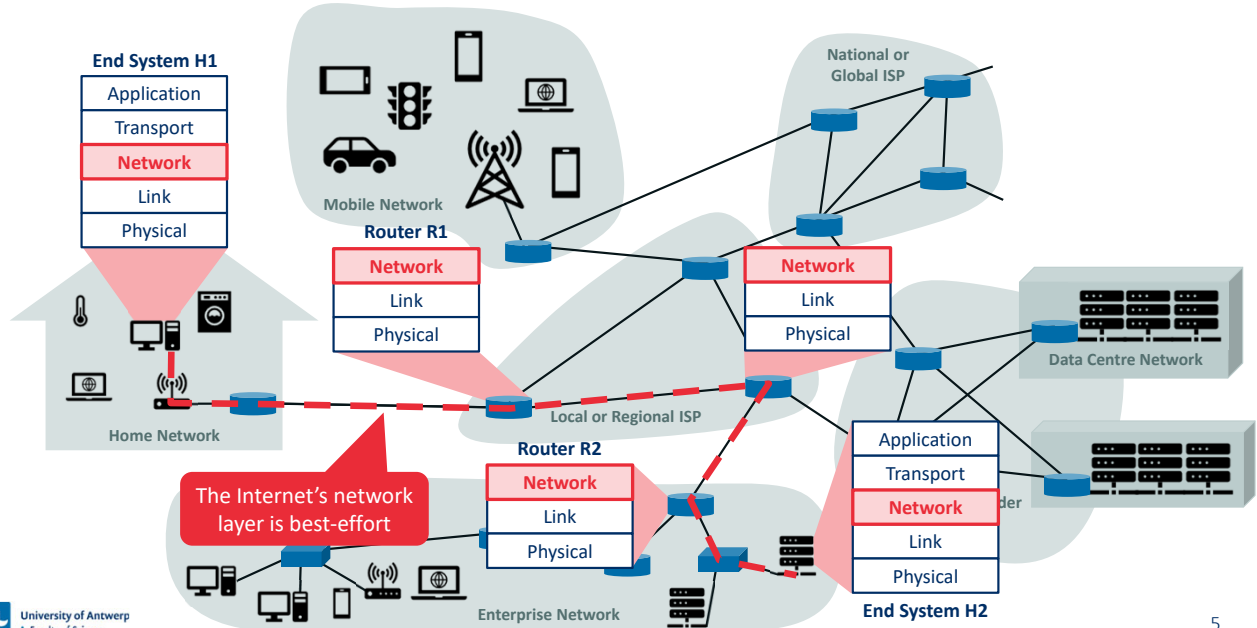
Network layer overview



Network Layer

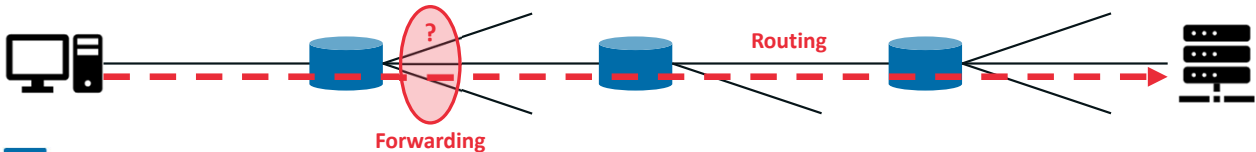
- Responsible for moving network-layer packet from one host to another
- Network-layer packets are called **datagrams**
- Functions
 - Addressing (Internet Protocol (IP) versions 4 and 6)
 - Routing (many intra- and inter-domain routing protocols)
- The Internet's network layer is connectionless and unreliable

Network layer overview

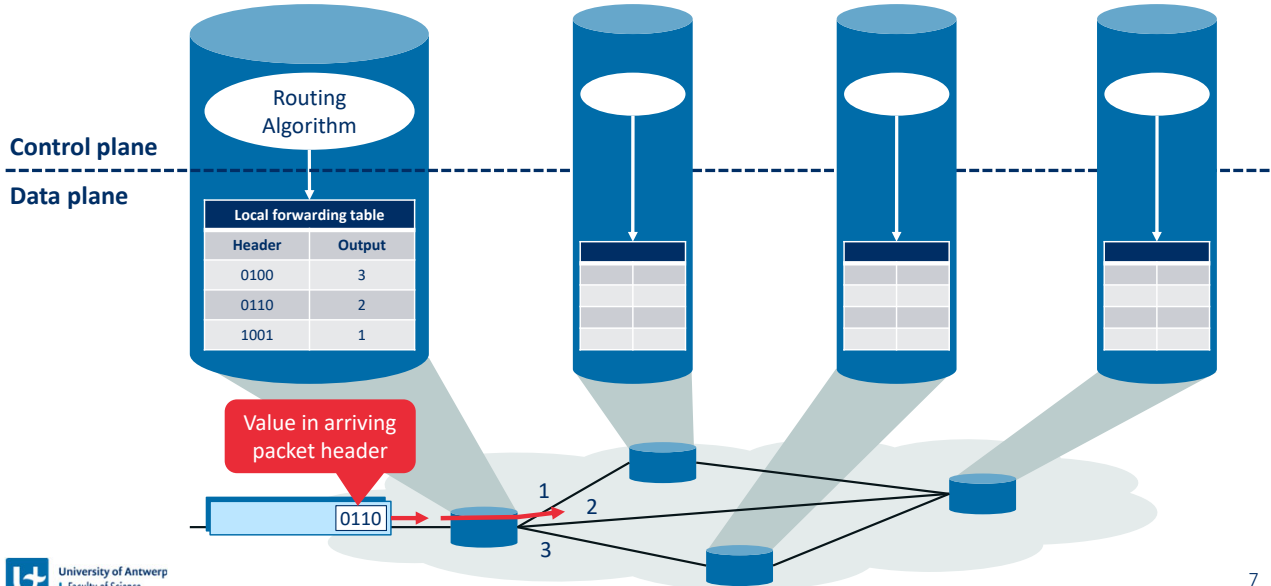


Forwarding and routing

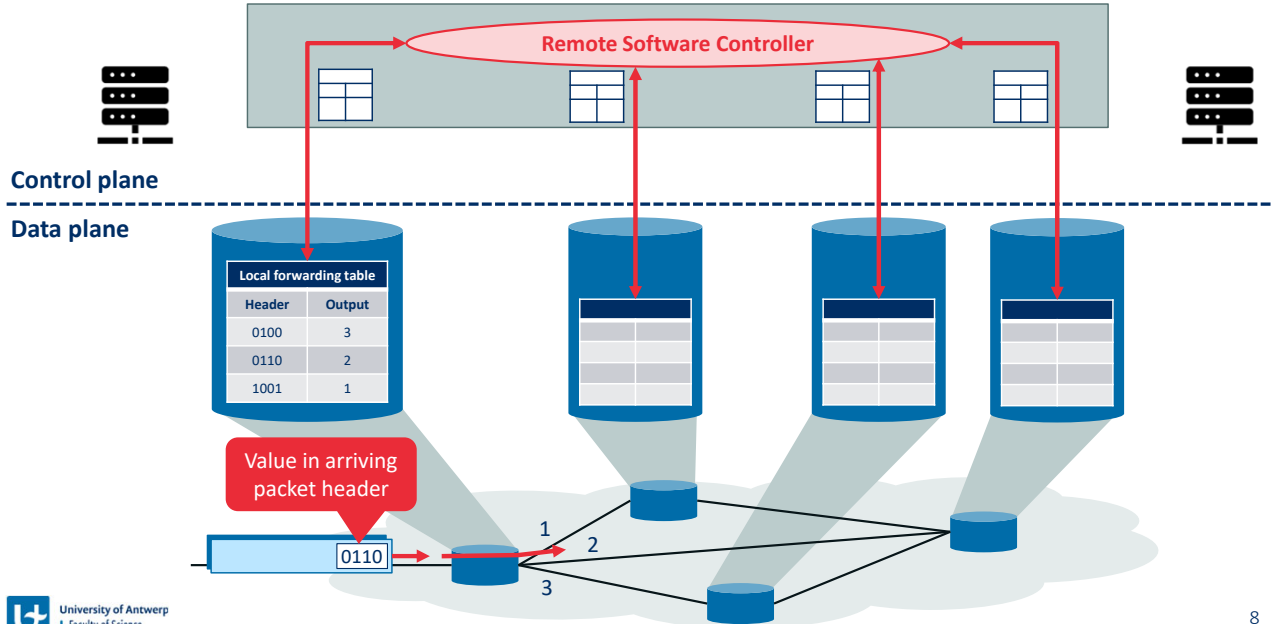
- **Forwarding** (data plane)
 - Move packet from the router's input link to the appropriate output link
 - Local decision taken at very short time frame (nanoseconds)
 - Typically implemented in hardware
- **Routing** (control plane)
 - Determine the route or path taken by packets from sender to receiver
 - Network-wide process that takes place on longer time scales (seconds)
 - Typically implemented in software



Traditional control plane



Software-defined networking (SDN) control plane



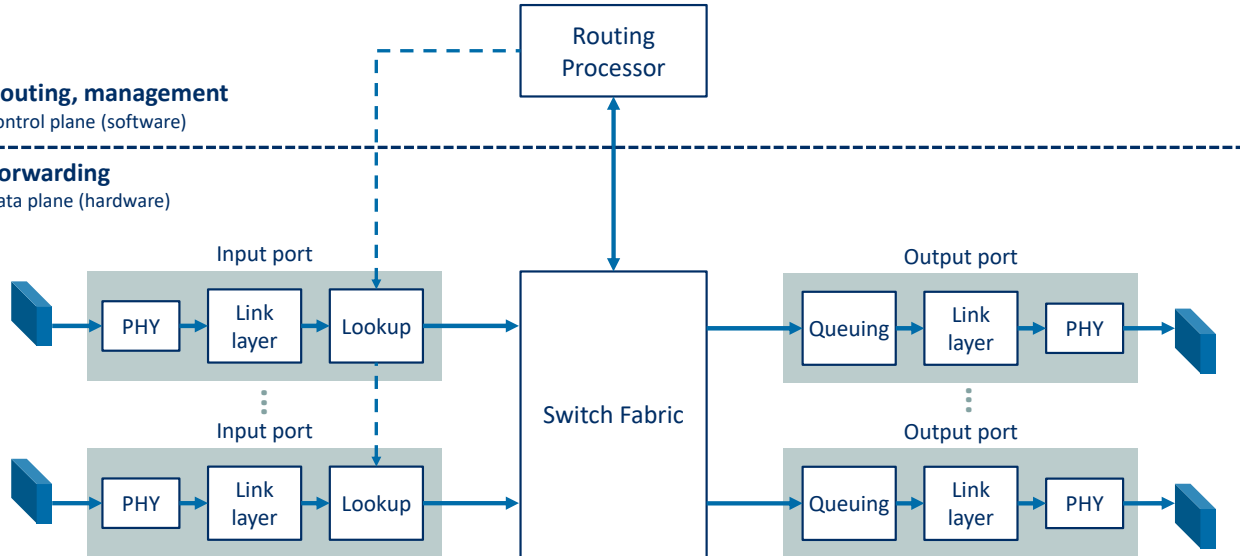
Dissecting a router

Routing, management

control plane (software)

Forwarding

data plane (hardware)





When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**



How much time does a router with 10 input ports of 100 Gbps have to process a single packet of 64 bytes?



5 ms

5 μ s

40 ns

5 ns

0.5 ns

Destination-based forwarding

- The forwarding table cannot contain an entry for each destination address (4 billion entries for IPv4!)
- Instead, forwarding table entries consist of **ranges**, represented by prefixes
- If a packet matches multiple prefixes, longest prefix matching is used

Destination address range		Prefix	Link Interface
11001000 00010111 00010000 00000000	11001000 00010111 00010111 11111111	11001000 00010111 00010*** *****	0
11001000 00010111 00011000 00000000	11001000 00010111 00011000 11111111	11001000 00010111 00011000 *****	1
11001000 00010111 00011001 00000000	11001000 00010111 00011111 11111111	11001000 00010111 00011*** *****	2
Otherwise		Otherwise	3

Exercise: Prefix matching



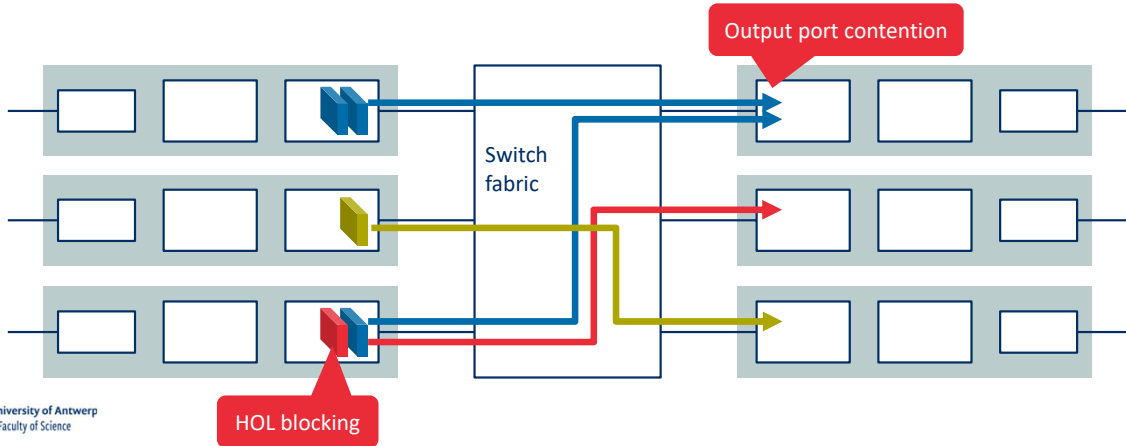
Exercise: On which interface is each of the packets forwarded?

Destination address range		Prefix	Link Interface
11001000 00010111 00010000 00000000	11001000 00010111 00010111 11111111	11001000 00010111 00010*** *****	0
11001000 00010111 00011000 00000000	11001000 00010111 00011000 11111111	11001000 00010111 00011000 *****	1
11001000 00010111 00011001 00000000	11001000 00010111 00011111 11111111	11001000 00010111 00011*** *****	2
Otherwise		Otherwise	3

- Packet 1: 11001000 00010111 00010110 10100001
- Packet 2: 11001000 00010111 00011000 10101010

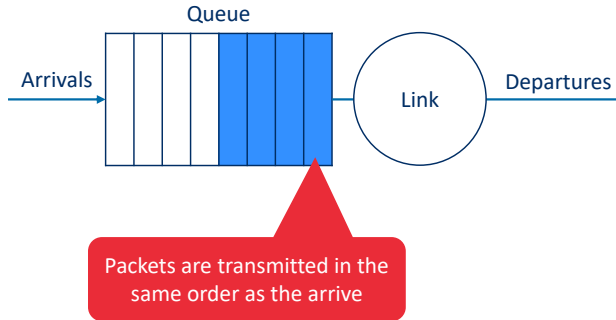
Queuing in routers

- When input or output queues (buffers) become full, **packet loss** will occur
 - **Head-of-the-line (HOL) blocking** occurs when a packet in an input queue must wait because it is blocked by another packet at the head of the line
 - **Output port contention** occurs when multiple packets are destined for the same output line at the same time



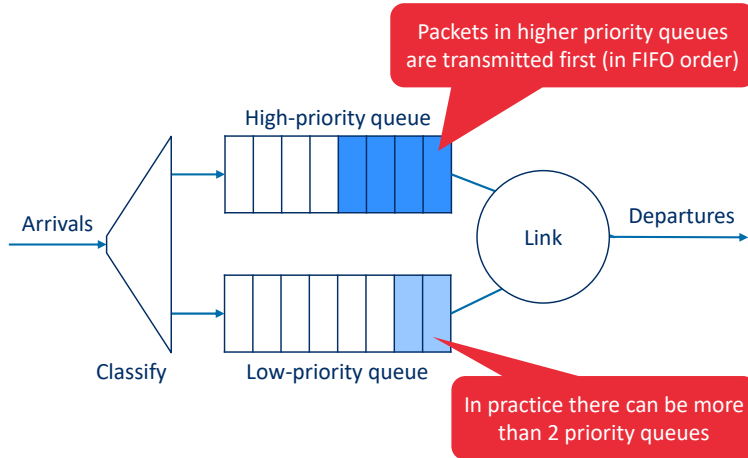
Packet scheduling techniques in router queues

First-in-first-out (FIFO)



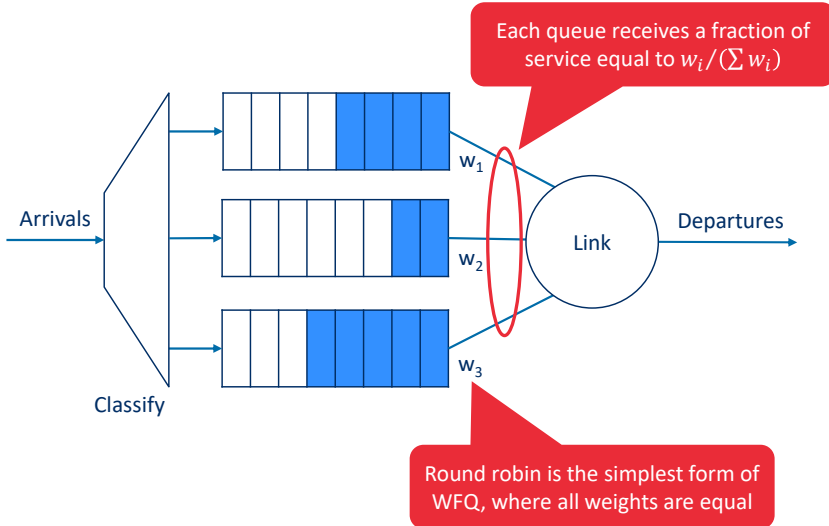
Packet scheduling techniques in router queues

Priority queuing



Packet scheduling techniques in router queues

Round robin and weighted fair queuing (WFQ)

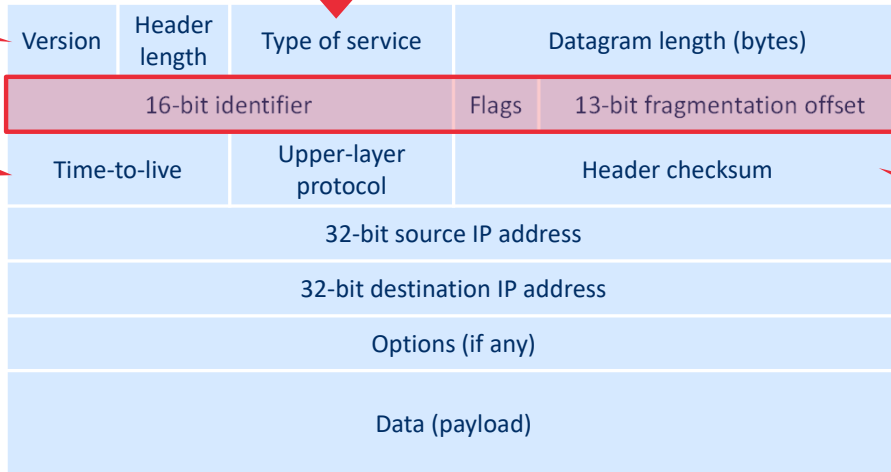


The Internet Protocol (IP)

IPv4 datagram format – RFC 791

Used for prioritization and Explicit Congestion Notification (ECN)

IPv4 or IPv6?



Used to avoid forwarding loops

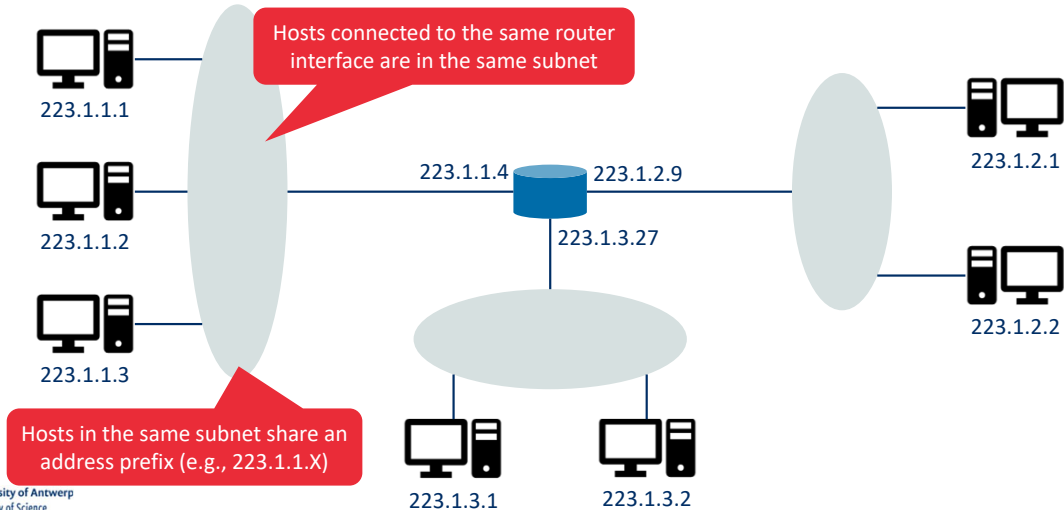
Used for IP fragmentation

Checked by each router

32 bits

IPv4 addressing

Between each host and physical link is an **interface**, that has its own IP address

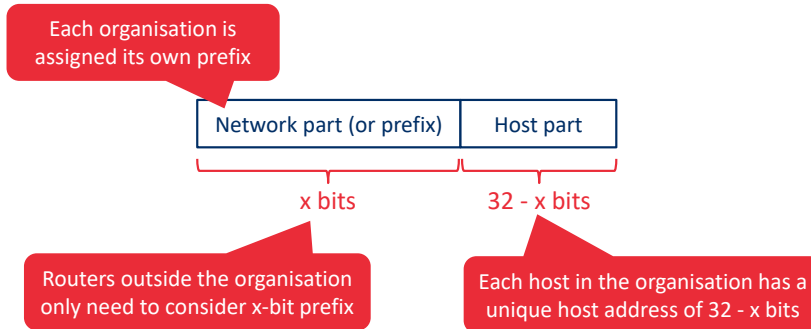


Classless Interdomain Routing (CIDR) – RFC 4632

- CIDR determines the Internet's address assignment strategy
- Each 32-bit IP address is split in two parts (x indicates the number of bits in the first part, also referred to as subnet)

a.b.c.d/x

- IP broadcast address: 255.255.255.255 (the message is delivered to all hosts in the subnet)





When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to +32 460 20 00 56



**What is the maximum number of interfaces in the subnet
223.1.3/29?**

0

3

8

128

256

2^{32}



When poll is active respond at **PollEv.com/jfamaey** Send **jfamaey** to **+32 460 20 00 56**



Which of the following addresses can **not** be used by an interface in the 223.1.3/29 network? Check all that apply.



223.1.3.6

223.1.2.6

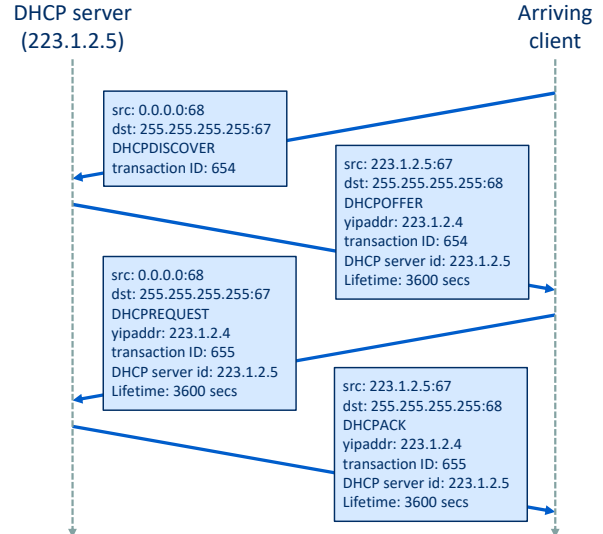
223.1.3.28

223.1.3.16

223.1.3.2

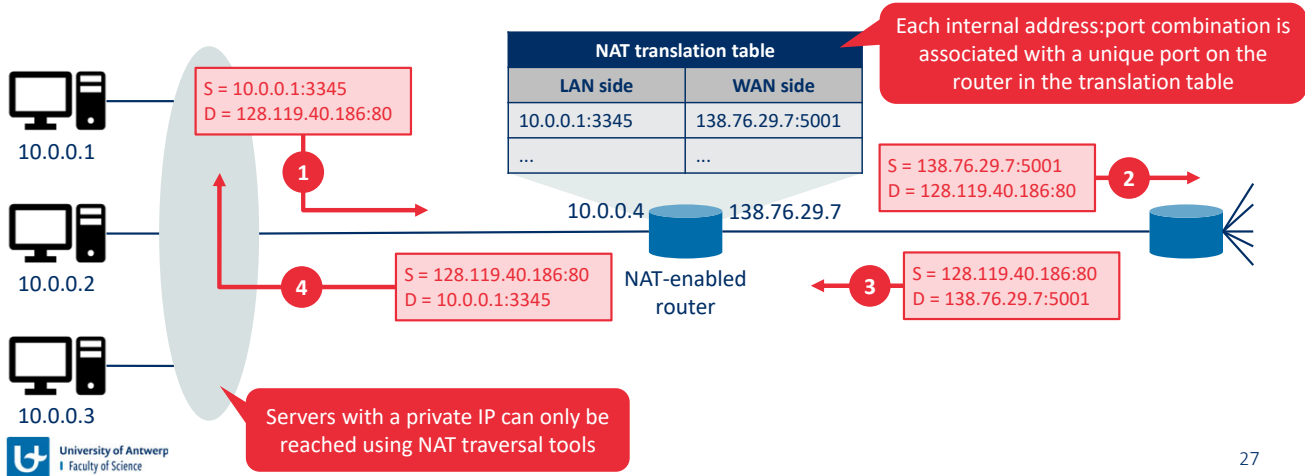
Dynamic Host Configuration Protocol (DHCP)

- A DHCP server offers several functions to hosts within a subnet
 - Automatically assign IP addresses to hosts
 - Provide additional information (subnet mask, IP address of first-hop router, local DNS server)
- DHCP consists of 4 steps
 - Discovery
 - Service offer(s)
 - Request
 - Acknowledgment
- An assigned IP address has a lifetime (i.e., amount of time it is valid), which can be extended with a renewal request



Network Address Translation (NAT)

- There are not enough IPv4 addresses to give every host in every network a unique IP address
- NAT offers a solution by associating a set of private addresses with a single public IP address



IPv6

- The last block of IPv4 addresses was assigned to a regional registry in February 2011
- IPv6 development began in the mid-1990s to prepare for this event
- Main improvements
 - Expanded addressing capabilities: IP address size is now 128 instead of 32 bits
 - Streamlined 40 byte header: Fixed length header is easier to process by routers
 - Flow labelling: IPv6 supports the concept of flows (i.e., packets that belong together at the application layer)

Version	Traffic class	Flow label	
Payload length		Next header	Hop limit
Source address (128 bits)			
Destination address (128 bits)			
Data (payload)			

An IPv6 address is usually represented as 8 hexadecimal numbers of 4 digits (16 bits) separated by a colon, e.g., FE80:CD00:0000:0CDE:1257:0000:211E:729C

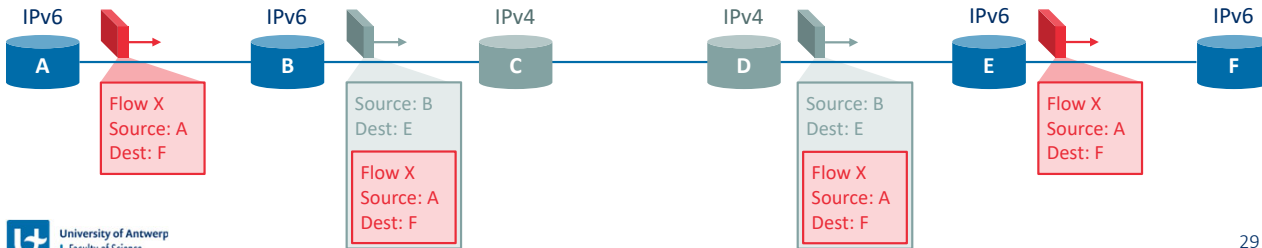
32 bits

Tunnelling: IPv4 and IPv6 co-existence

Logical view



Physical view



Routing algorithms

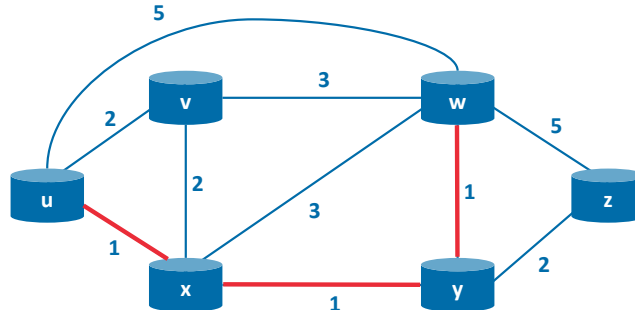
Routing algorithms

- **Goal:** Determine good paths (or routes), from senders to receivers, through the network of routers
- Routing problems are formulated using graph theory
 - A graph $G = (N, E)$ consists of a set of nodes (also called vertices) N and a set of edges E
 - In network-layer routing, the nodes are routers, and the edges are physical links between these routers
 - Each edge between any pair of nodes (x, y) has a cost $c(x, y)$ associated with it (e.g., representing its length, monetary cost)
 - A routing algorithm tries to find the **least-cost path** of edges $(x_1, x_2, \dots, x_p)$ between the source x_1 and destination x_p

Example:

Least-cost path from u to w is (u, x, y, w)

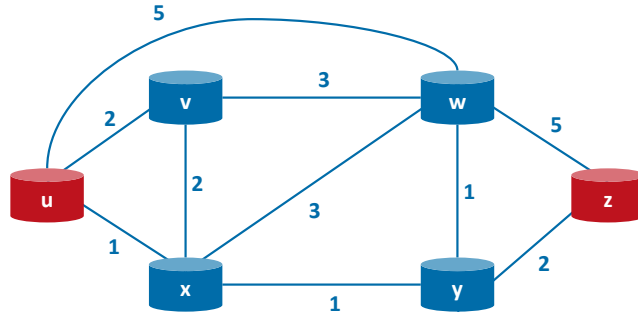
Cost: $c(u, x) + c(x, y) + c(y, w) = 3$



Exercise: Find the least-cost path



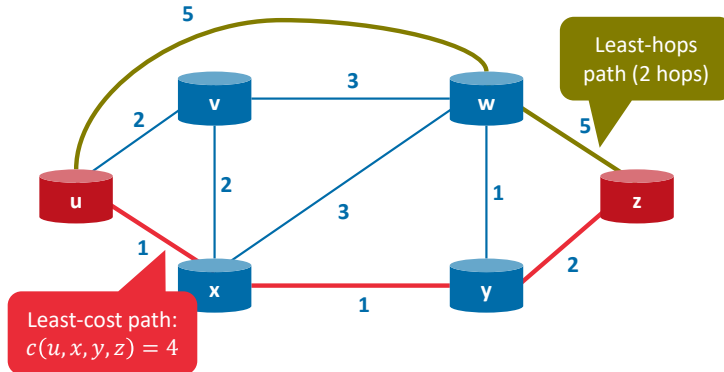
Exercise: What is the **least-cost path** from node u to node z? What is the **least-hops path**?



Solution: Find the least-cost path



Exercise: What is the **least-cost path** from node u to node z? What is the **shortest path**?



Classification of routing algorithms

Centralized (use complete global knowledge)

Static (routes change very slowly over time)

Load-sensitive (adapt to changing traffic load)

Decentralized (iterative, distributed)

Dynamic (routes adapt to real-time data)

Load-insensitive (link cost unaffected by load)

The (centralized) link-state (LS) routing algorithm

- Centralized routing algorithm that assumes the entire network topology and edge costs are known
- Can apply known least-cost path algorithms (e.g., Dijkstra's algorithm)
- Notations
 - N : set of all nodes
 - s : source node
 - $D(v)$: cost of the currently known least-cost path from the source node s to destination v
 - $p(v)$: previous node along the current least-cost path from the source s to v
 - N' : subset of nodes for which the least-cost path from the source s is known

- **Dijkstra's algorithm:**

Initialization: $N' = \{s\}$

$\forall v \in N: D(v) = c(s, v)$

$c(s, v) = \infty$ if s and v are not direct neighbours

Repeat:

$w = \underset{v \notin N'}{\operatorname{argmin}} D(v)$

$N' = N' \cup \{w\}$

$\forall$ neighbours v of $w \wedge v \notin N': D(v) = \min(D(v), D(w) + c(w, v))$

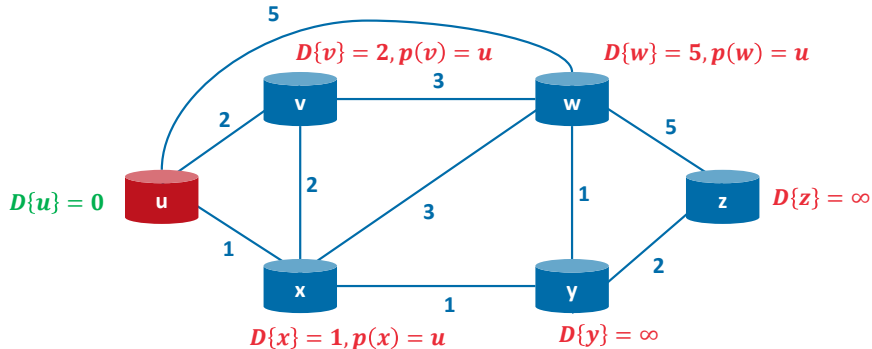
Until:

$N' = N$

Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Initialization

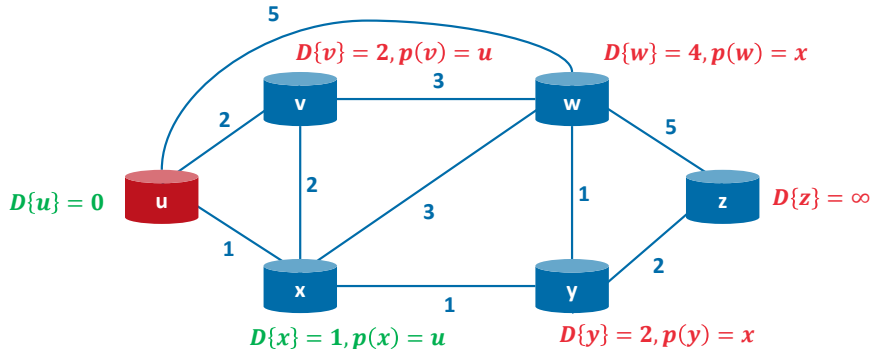
$$N' = \{u\}$$



Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 1

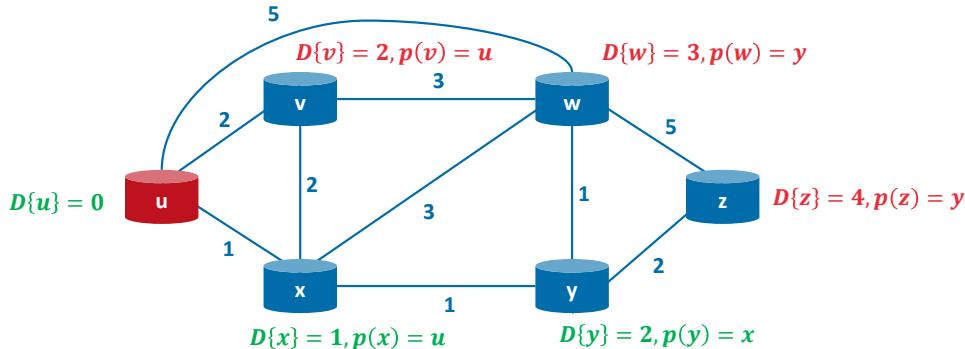
$$N' = \{u, x\}$$



Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 2

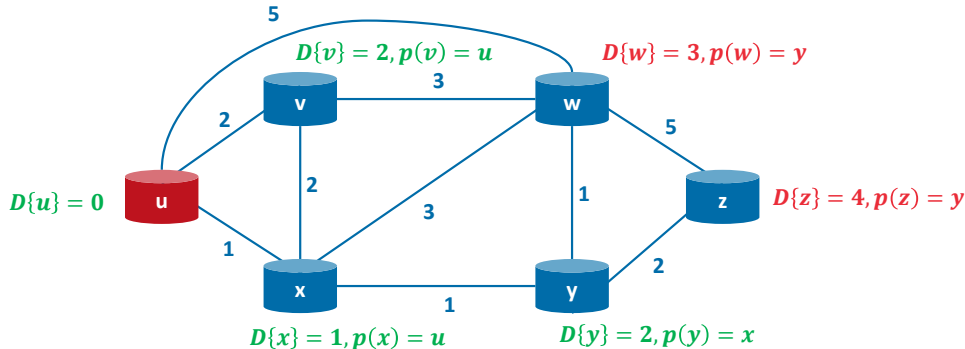
$$N' = \{u, x, y\}$$



Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 3

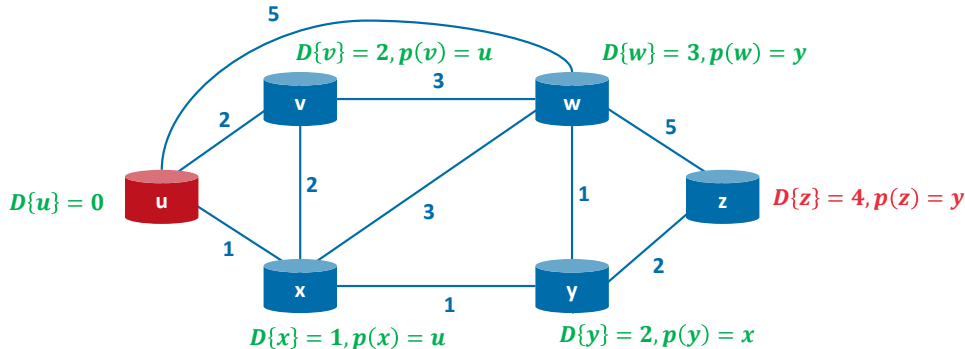
$$N' = \{u, x, y, v\}$$



Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 4

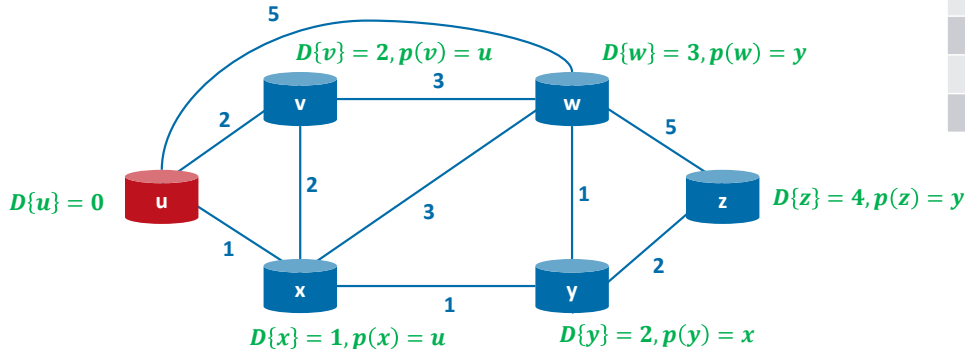
$$N' = \{u, x, y, v, w\}$$



Example: Dijkstra's algorithm (calculate least-cost paths from source u)

Iteration 5

$$N' = \{u, x, y, v, w, z\} = N$$



Forwarding table for node u :

Destination	Next Hop
v	v
w	x
x	x
y	x
z	x

Consider Dijkstra's link-state routing algorithm that is computing a least-cost path from node A to other nodes B, C, D, E, F. Which of the following statements are true?

Suppose nodes B, C, and D are in the set N' (selected by argmin on previous iterations). These nodes will remain in N' for the rest of the algorithm

0%

Following the initialization step, if nodes B and C are directly connected to A, then the least cost path to B and C will never change from this initial cost

0%

The values computed in the vector $D(v)$, the currently known least cost of a path from A to any node v , will always decrease in the following iterations

0%

The values computed in the vector $D(v)$, the currently known least cost of a path from A to any node v , will never increase in the following iterations

0%

In the initialization step, the initial cost from A to each of the destinations is initialized to either the cost of a link directly connecting A to a direct neighbor, or infinity otherwise

SEE MORE 

0%

What is the computational complexity of the "naive" Dijkstra's algorithm (given a network of n nodes and m edges)?

$O(\log(m))$

0%

$O(n*m)$

0%

$O(n*\log(m))$

0%

$O(n^2)$

0%

Computational complexity of naive Dijkstra's algorithm

Initialization: $N' = \{s\}$
 $\forall v \in N: D(v) = c(s, v)$

Repeat:

$$w = \operatorname{argmin}_{v \notin N'} D(v)$$

In iteration i , we need
to search $n-i$ nodes

$$N' = N' \cup \{w\}$$

$$\forall \text{ neighbours } v \text{ of } w \wedge v \notin N': D(v) = \min(D(v), D(w) + c(w, v))$$

Until:

$$N' = N$$

Total number of nodes visited is

$$(n-1) + (n-2) + \dots + 1 = \frac{n \times (n-1)}{2} = O(n^2)$$

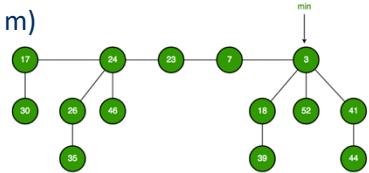
Total number of edges considered is m

$$\text{In worst case, } m = \frac{n \times (n-1)}{2} = O(n^2)$$

Computational complexity of Dijkstra's algorithm

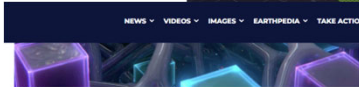
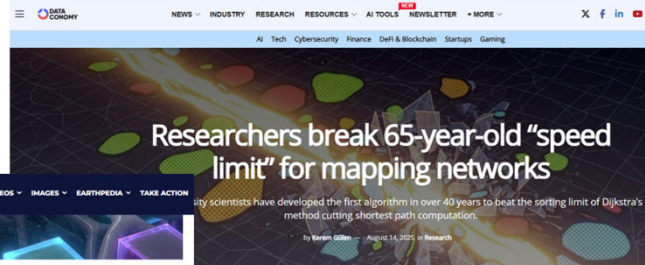
- Implementation using Fibonacci heap has lower complexity: $O(n * \log n + m)$

- $O(\log n)$ to extract minimum value
- $O(1)$ to update element
- Considered as “speed limit” for > 40 years
- Usage of binary heap instead of Fibonacci heap is widely used due to lower constants



- In 2025, scientists from Tsinghua University in Beijing developed a new algorithm: $O(m * \log^{2/3} n)$ complexity

- Becomes better than classical implementations for graphs with more than 10^{19} nodes (according to L. Castro et al. “Implementation and brief experimental analysis of the Duan et al. (2025) algorithm for single-source shortest paths”)



New algorithm finds the shortest path to any point in record time

By Jordan Joseph
Earth.com staff writer



Follow Earth on Google

Shortest path algorithms sit at the heart of modern graph theory and many of the systems that move people, data, and goods around the world.

After nearly seventy years of relying on the same classic approach, researchers have now created a faster way to solve this task.

New Method Is the Fastest Way To Find the Best Routes

18 |

A canonical problem in computer science is to find the shortest route to every point in a network. A new approach beats the classic algorithm taught in textbooks.

The (decentralized) distance vector (DV) routing algorithm

- Distributed routing algorithm where each node only relies on information obtained from neighbours
- Relies on the Bellman-Ford equation:

$$d_x(y) = \min_v \{c(x, v) + D_v(y)\}$$

where $D_x(y)$ is the least-cost distance from x to y

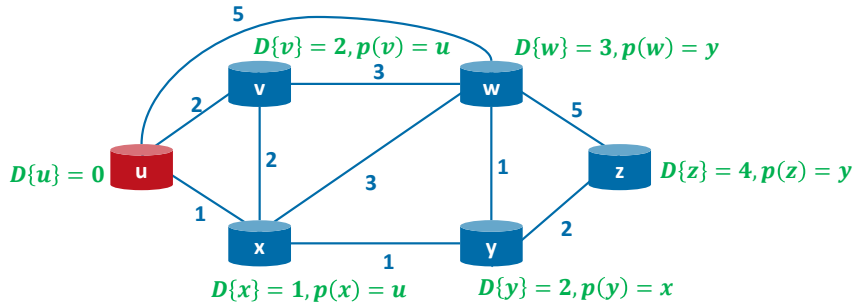
- The solution to the Bellman-Ford equation provides node x's forwarding table.
- Each node x keeps track of
 - An estimate distance vector $\mathbf{D}_x = [D_x(y): \forall y \in N]$
 - The cost $c(x, v)$ to each of its direct neighbours v
 - The distance vector of each of its neighbours v, $\mathbf{D}_v = [D_v(y): \forall y \in N]$

The (decentralized) distance vector (DV) routing algorithm

Relies on the Bellman-Ford equation:

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

where $d_x(y)$ is the least-cost distance from x to y



Homework: Check if the Bellman-Ford equation holds for source node u and destination node z in the previous example graph.

DV algorithm pseudocode (at each node x)

Initialization:

$\forall y \in N: D_x(y) = c(x, y)$

$\forall$ neighbours of w : send $D_x = [D_x(y): \forall y \in N]$ to w

Repeat:

(whenever link cost changes, or distance vector received from neighbour)

$\forall y \in N: D_x(y) = \min_v \{c(x, v) + D_v(y)\}$

set $v^* = \operatorname{argmin}\{c(x, v) + D_v(y)\}$ as next hop for dest y

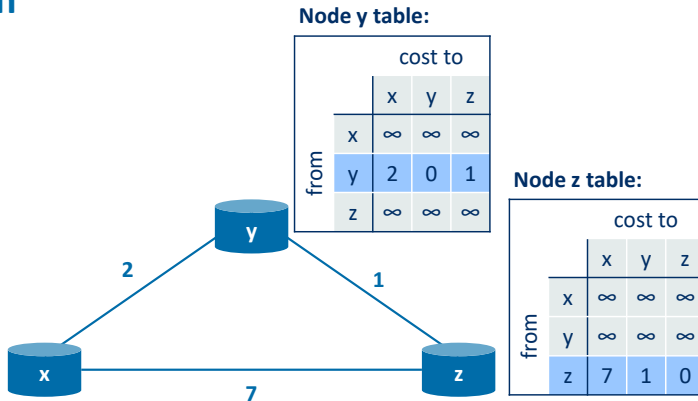
if $D_x(y)$ changed for any destination y :

$\forall$ neighbours of w : send $D_x = [D_x(y): \forall y \in N]$ to w

forever

Example: DV algorithm

Initialization



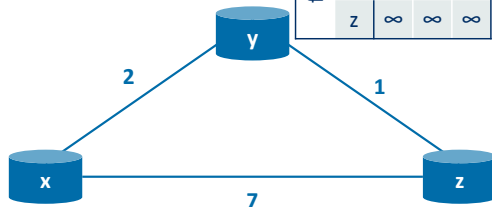
Node x table:

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

Example: DV algorithm

Step 1

Node x received D_y and D_z



Node y table:

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

Node z table:

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

Node x table:

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

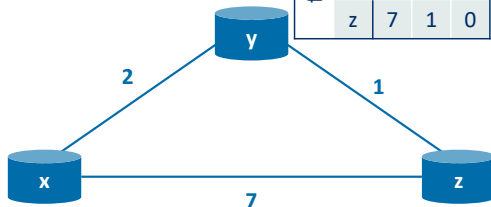
		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

$c(x, y) + D_y(z)$

Example: DV algorithm

Step 2

Node y receives D_x and D_z



Node y table:

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

Node z table:

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

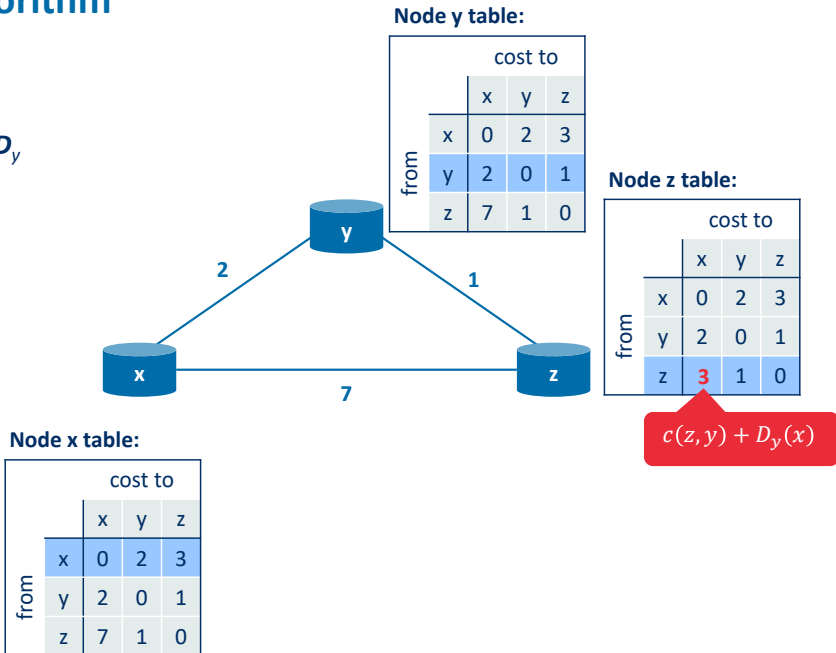
Node x table:

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

Example: DV algorithm

Step 3

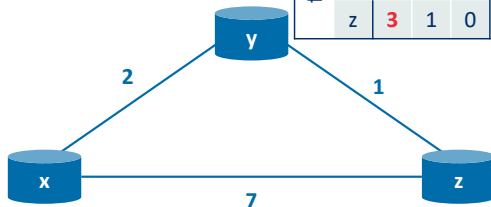
Node z receives D_x and D_y



Example: DV algorithm

Step 4

Node x and y receive D_z



Node y table:

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

Node z table:

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

Node x table:

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

Router A has two neighbors, B and C. The link costs are $A \rightarrow B = 2$ and $A \rightarrow C = 8$. Initially, A knows B can reach X with a cost of 10. Later, C sends an update saying it can reach X with a cost of 3. What is A's new routing decision for X?

A keeps its route through B because B was the first to report a path

0%

A switches to C because the total path cost through C is better than the path through B

0%

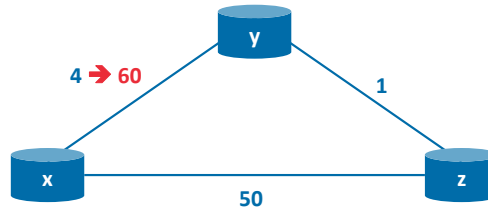
A keeps its route through B because the total path cost through B is better than the path through C

0%

A switches to C because the new path is more fresh

0%

Routing loops in DV due to link cost increase (count-to-infinity problem)



Node y table:

Before change:

		cost to			
		x	y	z	
from	x	0	4	5	
	y	4	0	1	
	z	5	1	0	

Step 1: cost change,
advertise D_y to x and z

		cost to			
		x	y	z	
from	x	0	4	5	
	y	6	0	1	
	z	5	1	0	

Routing loop!

Step 2: receive D_x and D_z

		cost to			
		x	y	z	
from	x	0	51	50	
	y	6	0	1	
	z	7	1	0	

Step 3: recalculate DVs,
advertise D_y to x and z

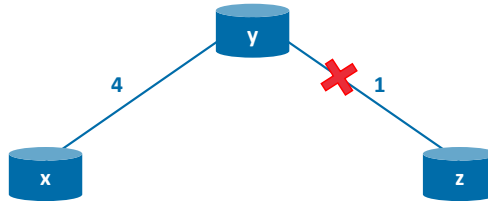
		cost to			
		x	y	z	
from	x	0	51	50	
	y	8	0	1	
	z	7	1	0	

...

Step 46: self-correction

		cost to			
		x	y	z	
from	x	0	51	50	
	y	51	0	1	
	z	50	1	0	

Routing loops in DV due to link loss (count-to-infinity problem)



Node y table:

Before loss:

		cost to			
		x	y	z	
from	x	0	4	5	
	y	4	0	1	
	z	5	1	0	

Step 1: link loss,
advertise D_y to x

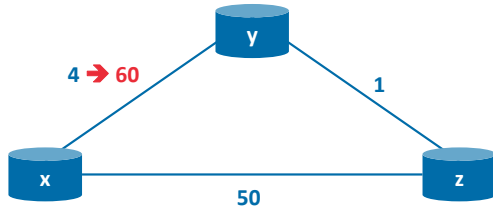
		cost to			
		x	y	z	
from	x	0	4	5	
	y	4	0	9	
	z	5	1	0	

Step 2: receive D_x ,
recalculate DVs

		cost to			
		x	y	z	
from	x	0	4	13	
	y	4	0	9	
	z	5	1	0	

...

Poisoned reverse



Solution – Poisoned Reverse:

If a node x routes through y to get to z, then x will always tell y that $D_x(z)=\infty$

Node y table

Before change:

		cost to			
		x	y	z	
from	x	0	4	∞	
	y	4	0	1	
	z	∞	1	0	

Node y table

Step 1: cost change

		cost to			
		x	y	z	
from	x	0	4	∞	
	y	60	0	1	
	z	5	1	0	

Advertisement D_y

Step 2: Advertise D_y to z

		cost to			
		x	y	z	
from	y	60	0	1	

Node z table

Step 3: receive D_y ,
recalculate DVs

		cost to			
		x	y	z	
from	x	0	∞	50	
	y	60	0	1	
	z	50	1	0	

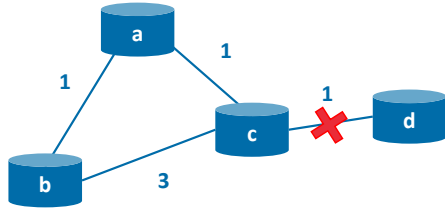
Node y table

Step 4: receive D_z ,
self-correction

		cost to			
		x	y	z	
from	x	0	51	50	
	y	51	0	1	
	z	50	1	0	

Poisoned
reverse!

Poisoned reverse



Poisoned Reverse **does not always work**

If a node y routes through z to get to x, then y will always tell z that $D_y(x) = \infty$

Node c table

Before change:

		cost to				
		a	b	c	d	
from	a	0	1	1	∞	
	b	1	0	2	3	
	c	1	2	0	1	
	d	∞	∞	1	0	

Node c table

Step 1: link loss,
update DVs

		cost to				
		a	b	c	d	
from	a	0	1	1	∞	
	b	1	0	2	3	
	c	1	2	0	6	
	d	∞	∞	1	0	

Node a table

Step 2: update DVs
at node a after D_c

		cost to				
		a	b	c	d	
from	a	0	1	1	7	
	b	1	0	∞	∞	
	c	1	∞	0	6	

Node b table

Step 3: update DVs at
node b after D_c and D_a

		cost to				
		a	b	c	d	
from	a	0	1	1	7	
	b	1	0	2	8	
	c	1	2	0	6	

Node c table

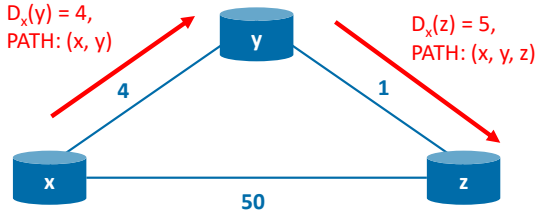
Step 4: update DVs at
node c after D_a and D_b

		cost to				
		a	b	c	d	
from	a	0	1	1	∞	
	b	1	0	2	8	
	c	1	2	0	11	
	d	∞	∞	1	0	

...

Path-vector routing

- **Key idea:** advertise the entire path to the destination node along with distance metric
- **Advantages:**
 - avoid count-to-infinity problem due to fast loop detection (node cannot be twice in the path)
 - support of flexible routing policies based on local preference of one path over another



How do Link-State and Path-Vector protocols differ in the way they ensure a path is loop-free?

Link-State trusts the neighbor's calculation, while Path-Vector uses a "Hello" protocol to verify the neighbor is still alive

0%

Link-State relies on "Poisoned reverse" to hide loops, while Path-Vector uses timeouts to wait for the network to stabilize

0%

Both protocols use the same mechanism: they count the number of hops and declare the path "loop-free" as long as the total distance is less than an arbitrarily chosen big threshold

0%

Link-State provides each router with a complete map of the network to calculate its own shortest path; Path-Vector includes a list of all visited nodes (IDs) in the update so a router can reject any path that contains its own...

0%

Routing protocols summary

- In **Link-State protocols**, all routers have complete information about connectivity and link costs. Routers calculate shortest paths using Dijkstra's algorithm. The algorithm ensures that the chosen paths are loop-free.
- In **Distance-Vector protocols**, routers rely on the knowledge provided by their neighbors. Each router advertises the vector of current shortest distances to all reachable nodes in the network. Special mechanisms, such as Poisoned Reverse, are used to avoid loops. To overcome count-to-infinity, they need a threshold value.
- In **Path-Vector protocols**, a list of all visited nodes is advertised together with the distance vector. Routers reject the paths that already contain their own ID.

Routing on the Internet: OSPF & BGP

Routing on the Internet happens on two levels

- The Internet is split into **autonomous systems** (ASs)
 - Each AS consists of a group of routers that are under the same administrative control (e.g., owned by one ISP)
 - Each AS has a globally unique autonomous system number (ASN)
- Two different types of routing algorithms
 - Within an AS (**intra-AS**): Interior Gateway Protocol (IGP), such as Open Shortest Path First (OSPF)
 - Across ASs (**inter-AS**): Exterior Gateway Protocol (EGP), such as Border Gateway Protocol (BGP)
- Advantages of a two-stage approach
 - **Scalability**: The number of routers on the Internet is too big for a single routing algorithm to handle
 - **Administrative autonomy**: Each ISP wants to enforce its own policies on the routing algorithms it uses

Interior Gateway Protocols (IGPs)

- Open Shortest Path First (OSPF)
 - Based on link-state routing and Dijkstra's shortest path algorithm
 - Standardized as RFC 2328 (IPv4) and RFC 5340 (IPv6)
 - Widely used in large enterprise networks
- Intermediate System to Intermediate System (IS-IS)
 - Conceptually very similar to OSPF (also based on link-state routing and Dijkstra's shortest path algorithm)
 - Standardized as ISO/IEC 10589:2002
 - Widely used in large service provider networks
- Enhanced Interior Gateway Routing Protocol (EIGRP)
 - Based on distance-vector routing
 - Developed as a proprietary protocol by Cisco, but eventually standardized as RFC 7868 in 2016

Open Shortest Path First (OSPF)

- Based on link-state routing and Dijkstra's shortest path algorithm
 - Routers **flood** link-state information [i.e., current knowledge of the entire AS] **to all other routers in the autonomous system**, not only direct neighbours
 - Link-state is advertised when a link changes (cost change, comes up, or goes down)
 - Link-state is also advertised periodically (at least once every 30 minutes)
 - Link availability is checked by sending periodic HELLO messages to all direct neighbours
- Works directly on top of IP, i.e., none of the transport layer protocols are used. OSPF should take care of reliable delivery of its messages on its own.
- Link weights are manually configured by the administrator
- Other features
 - Security: Exchanges between OSPF routers can be authenticated (when IPv6 is used, can rely on IP Authentication Header)
 - Multi-path: If multiple same-cost paths exist, they can be used simultaneously to each carry part of the traffic
 - Multicast support
 - Hierarchical division of AS into multiple independent routing areas

OSPF initialization and link-state database synchronization

RouterID = 0.0.0.1

RouterID = 0.0.0.2

Not the same as IP address



Hello (srcID = 0.0.0.1,
Active Neighbor = None)

Hello (srcID = 0.0.0.2,
Active Neighbor = **0.0.0.1**)

dst IP address: 224.0.0.5 (IPv4) or ff02::5 (IPv6)
multicast to all OSPF routers in the same
broadcast area (to routers b and c)

DB Description (srcID = 0.0.0.1,
seq = x, Init, Master, More)

DB Description is unicast

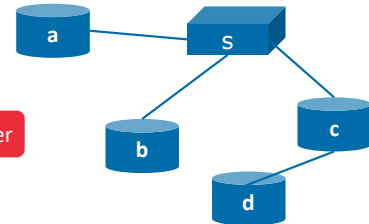
DB Description (srcID = 0.0.0.2,
seq = y, Init, **Master**, More)

My RouterID is higher => I will be Master

DB Description (srcID = 0.0.0.1,
seq = y, **Slave**, More)

Same seq => previous packet is acknowledged.
If unexpected seq: repeat previous packet

...



OSPF initialization and link-state database synchronization

RouterID = 0.0.0.1

RouterID = 0.0.0.2



DB Description (srcID = 0.0.0.2,
seq = z, **Master**, NoMore)

DB Description (srcID = 0.0.0.1,
seq = z, **Slave**, NoMore)

LS Request (srcID = 0.0.0.2, list of
requested LSAs)

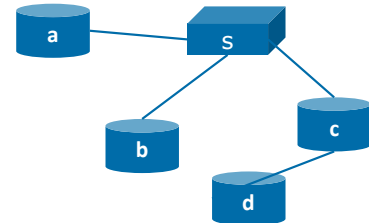
LS Update (srcID = 0.0.0.1,
requested LSA content)

LS Acknowledge (srcID = 0.0.0.2,
list of acknowledged LSAs)

More flag is set to False => last piece

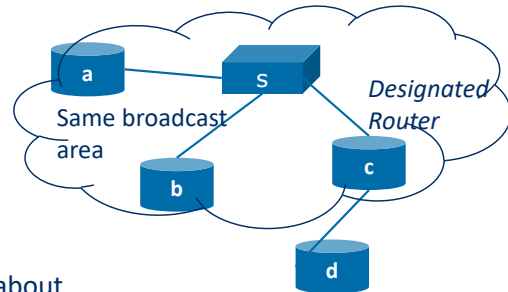
DB Description contains only headers
of Link State Advertisements (LSAs).
The content needs to be requested

LS Request will be sent from a to b
as well



OSPF Link-State Advertisements (LSAs)

- **Router LSA** contains information about interfaces of the router. For each interface, it provides a RouterID of the neighboring router connected through this interface and link cost (metric).
In *OSPFv2*, also provides the IP address of the Designated Router in the broadcast area.
- **Network LSA** is generated by designated router, and contains information about all attached routers in this broadcast area. Designated router is responsible for other routers in this area.
In *OSPFv2*, also provides the network mask.
- **Intra-area prefix LSA** (*OSPFv3* only) contains information about network IP addresses (prefixes) reachable through a router.



Which of the following statements regarding OSPF behavior and logic are correct?

OSPF is a distance-vector protocol that relies on "routing by rumor" to understand the network topology

0%

Database Description (DBD) packets are sent via unicast because the neighbor's specific IP address was already learned from Hello packets

0%

The "Slave" router in a Database Description (DBD) exchange is prohibited from sending its own unique Link-State Advertisements

0%

OSPF routers only learn about networks that are directly connected to their immediate neighbors

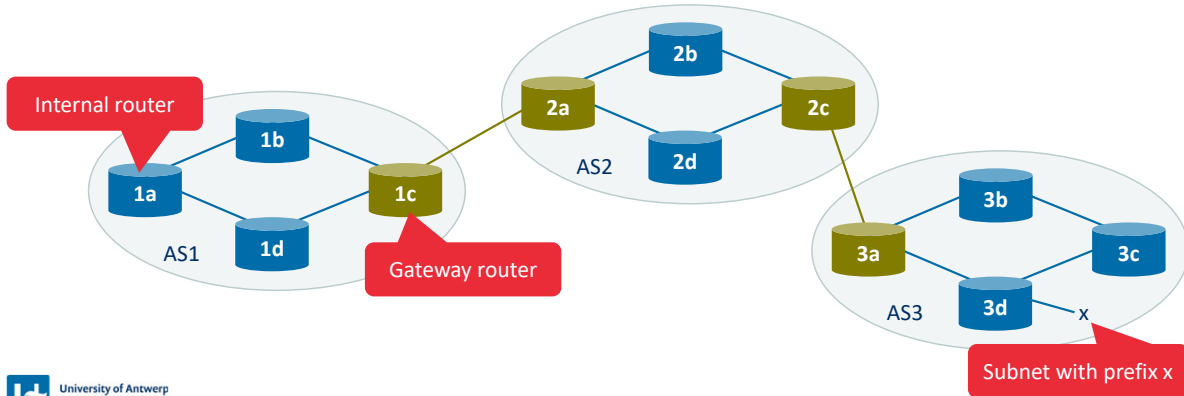
0%

A Designated Router (DR) is elected on broadcast segments primarily to reduce the total number of neighbor adjacencies and redundant traffic

0%

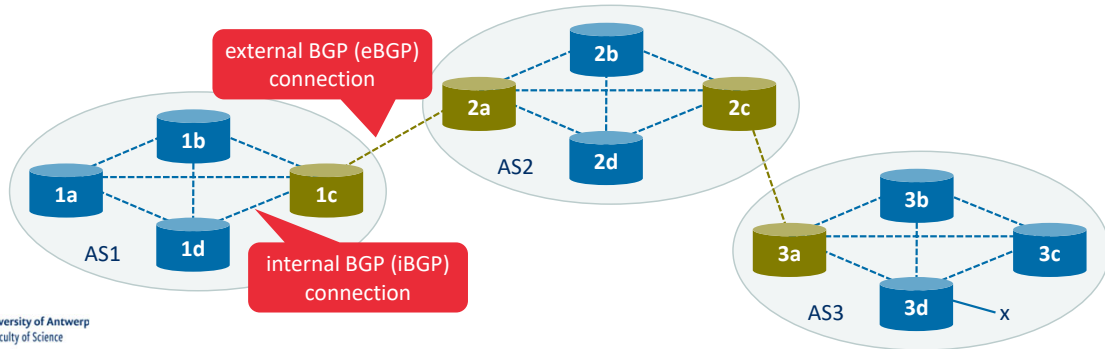
Border Gateway Protocol (BGP)

- BGP is used for routing between all ASs on the Internet
- Decentralized and asynchronous, based on **path-vector** routing
- Relies on TCP for delivery of its messages
- Routing is done based on CIDR prefixes (e.g., 138.16.68/22). Summary prefixes can be advertised instead of many specific prefixes from the AS (e.g., one /22 prefix instead of multiple /24 prefixes)

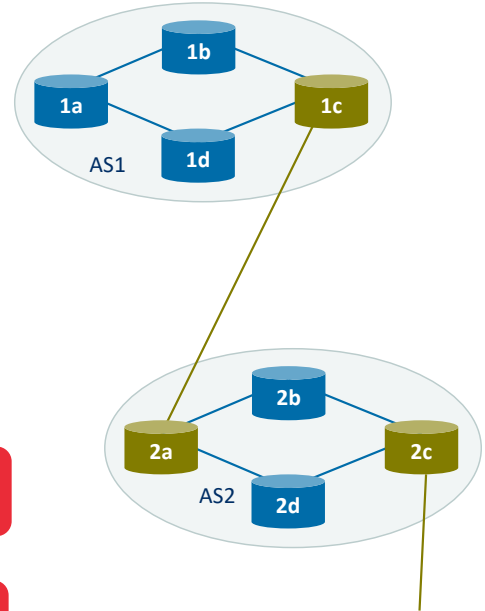
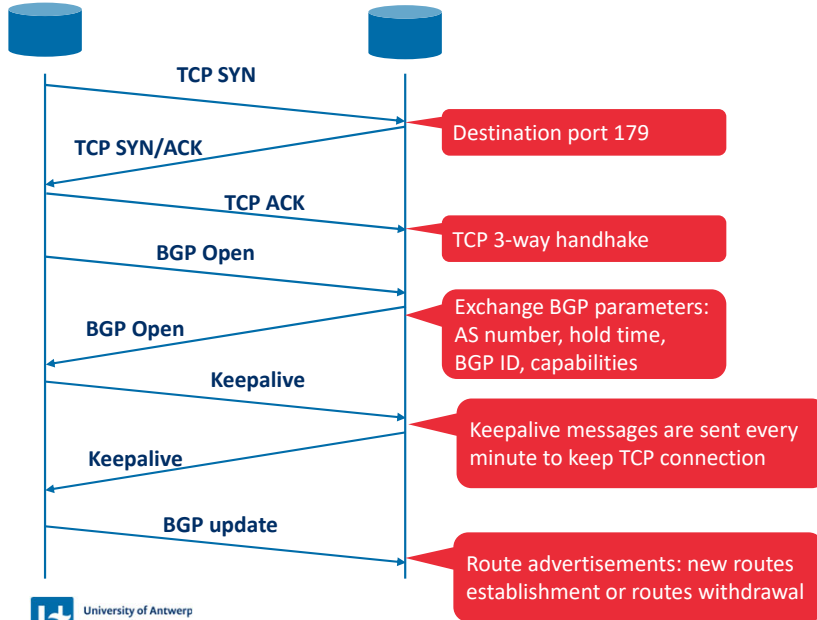


BGP connections

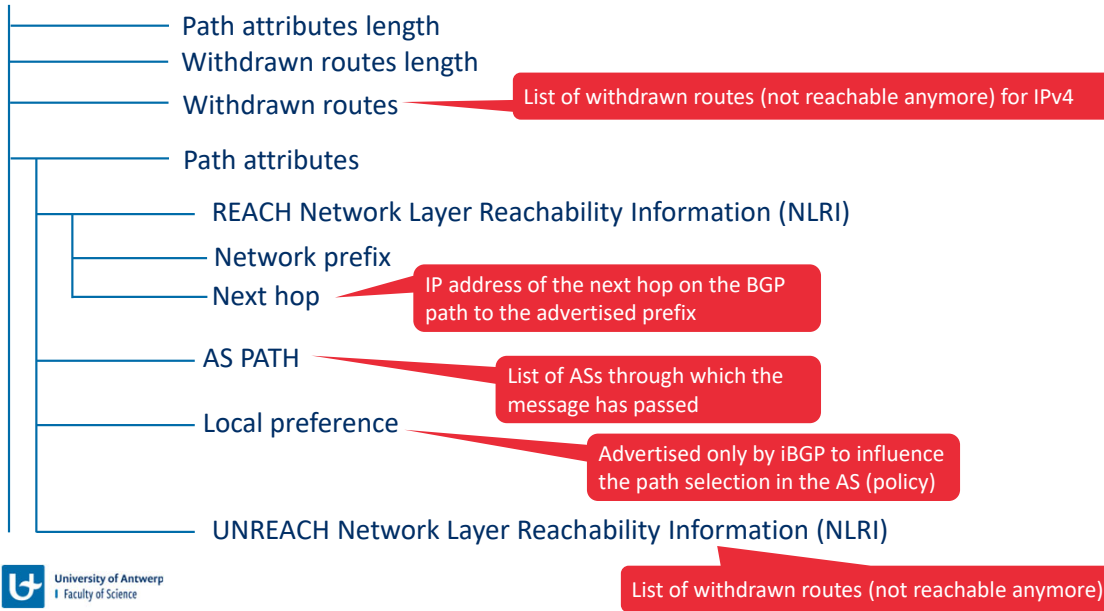
- How is information about subnet x in AS3 advertised to the Internet?
- Routers maintain a semi-permanent TCP connection on port 179 to exchange BGP information
 - **iBGP connection**: TCP connection between two routers within the same AS
 - **eBGP connection**: TCP connection between two routers in different ASs
- Within an AS, all routers maintain an iBGP connection to each other (in a full mesh or using Route Reflectors), while eBGP connections are only maintained between direct neighbours in different ASs.



BGP initialization

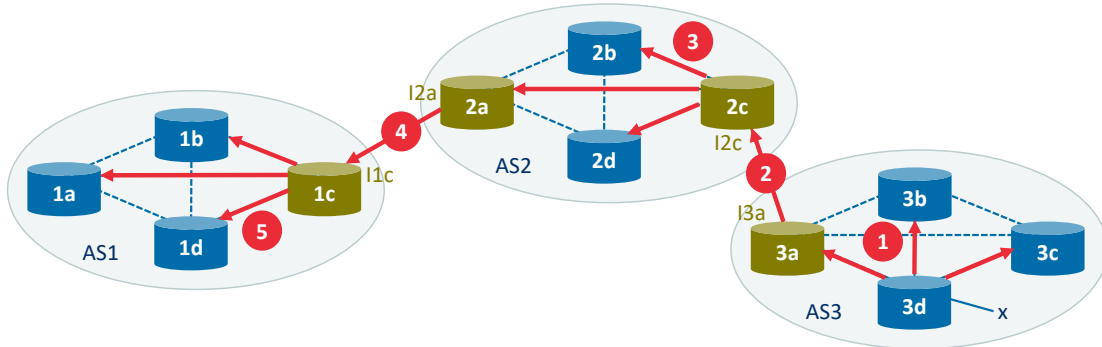


BGP Update messages



BGP advertisement messages

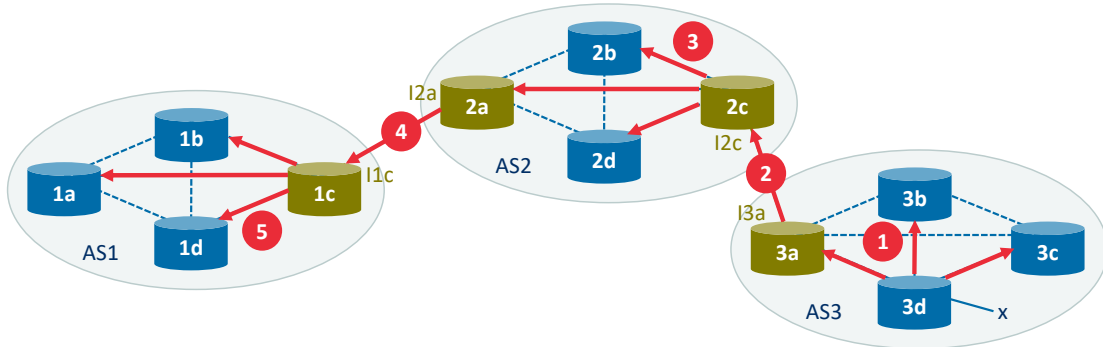
- BGP advertisements (also called routes) consist of an address prefix and several attributes
 - AS-PATH: List of ASs through which the message has passed
 - NEXT-HOP: IP address of the router interface that begins the AS-PATH
- **Example (advertising subnet x):**



Exercise: BGP advertisement messages



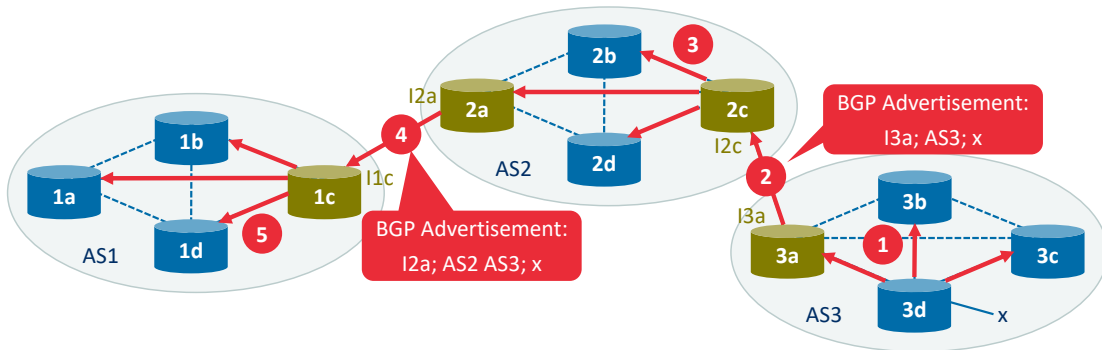
Exercise: What is the content of the BGP advertisement messages sent at steps 2 and 4, assuming a structure “NEXT-HOP; AS-PATH; prefix”?



Solution: BGP advertisement messages



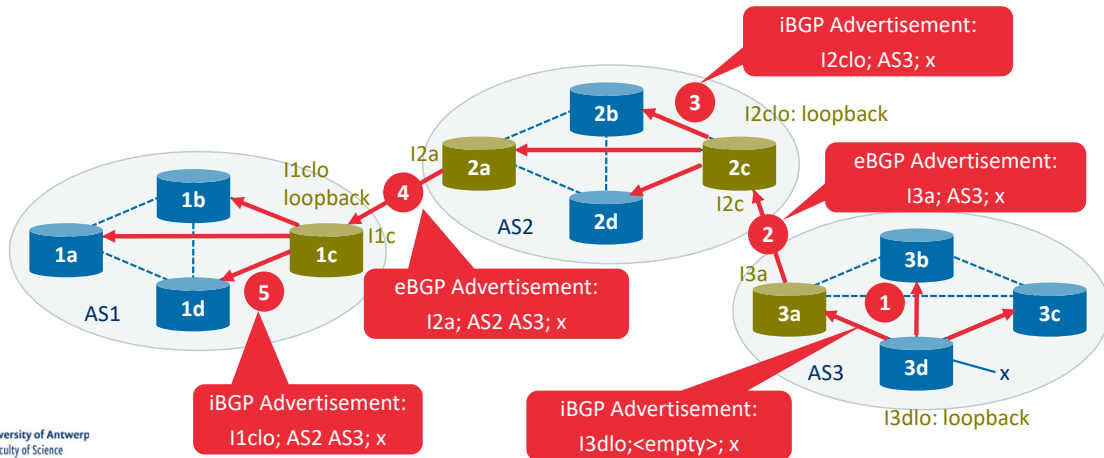
Exercise: What is the content of the BGP advertisement sent at steps 2 and 4, assuming a structure “NEXT-HOP; AS-PATH; prefix”?



BGP advertisement messages: iBGP next hop self

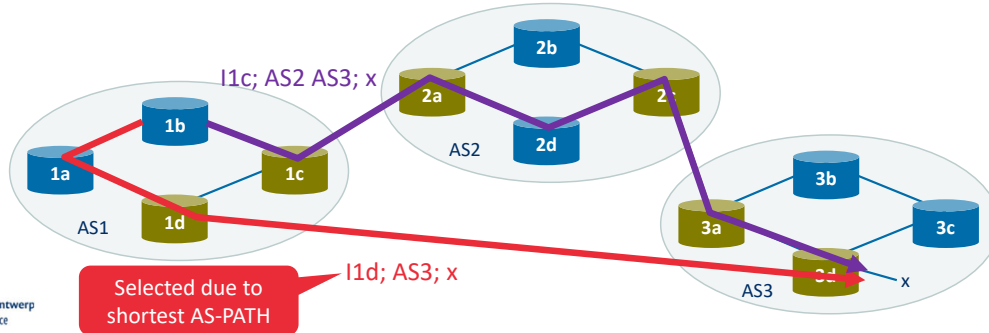
IP address of interface I3a is unknown inside AS2. To be able to route to next hop inside AS2, iBGP substitutes IP address of the next hop with internal IP address.

A good practice is to use loopback interface.



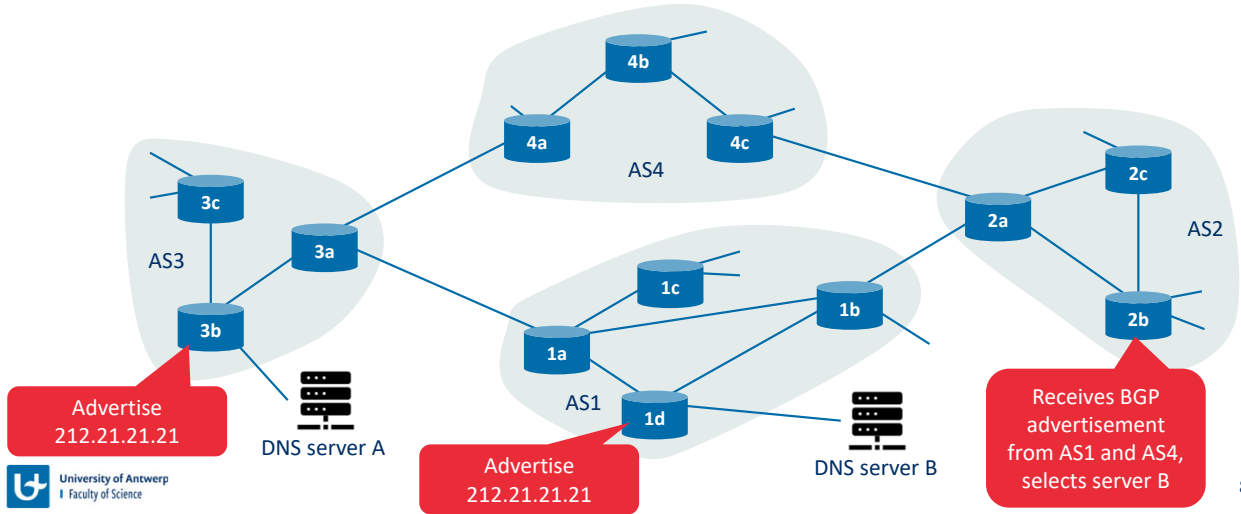
BGP route-selection algorithm

- BGP sequentially invokes elimination rules until only 1 route remains
 1. If any routes have the **local preference value** set, the route with the highest local preference is selected
 2. From the remaining routes, the one with the **shortest AS-PATH** is selected
 3. From the remaining routes, the one with the **closest NEXT-HOP** router is selected (determined using intra-AS routing, e.g., OSPF)
 4. If more than 1 route remains, a route is selected based on the BGP identifier
- **Example:** Which of the 2 routes would be selected?



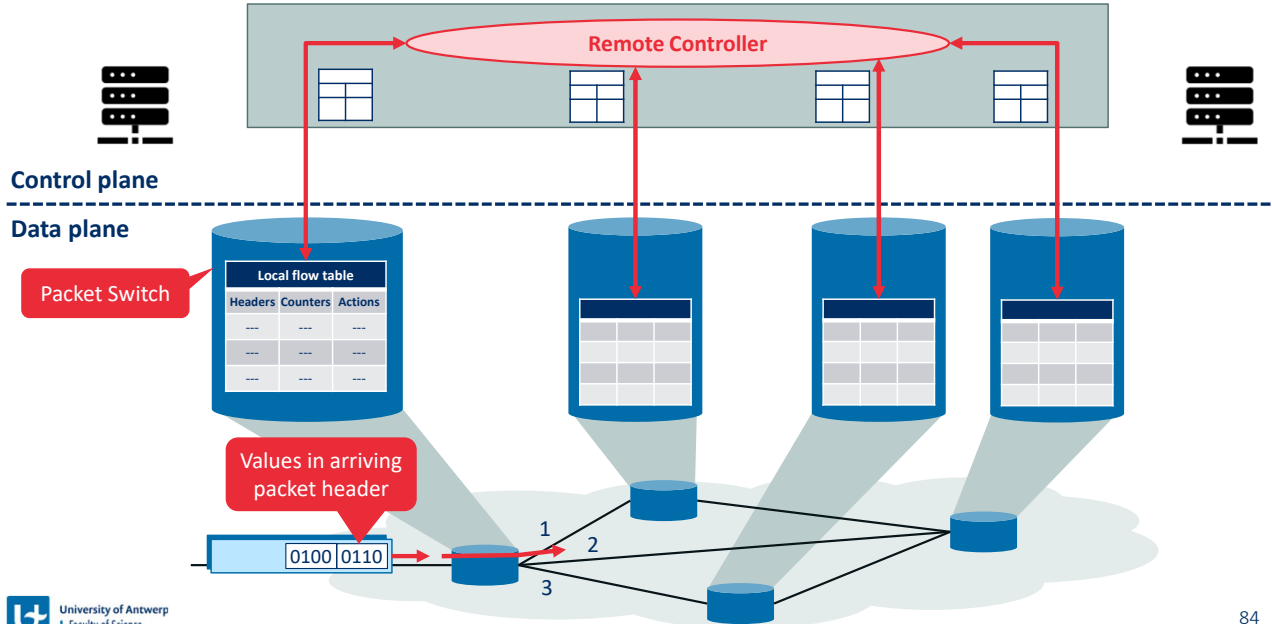
IP-Anycast

- BGP can be used to advertise the same IP address for multiple servers, allowing routers to automatically select the nearest one (e.g., DNS server replication)



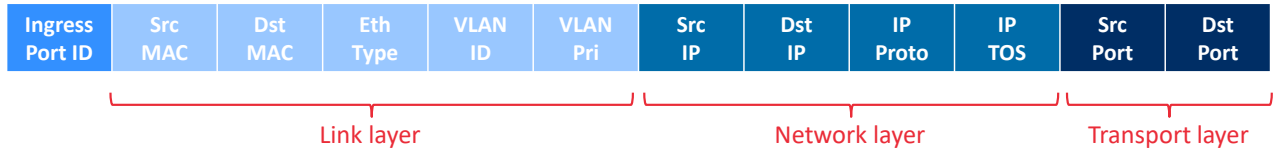
Software defined networking (SDN)

SDN control plane with match-plus-action table



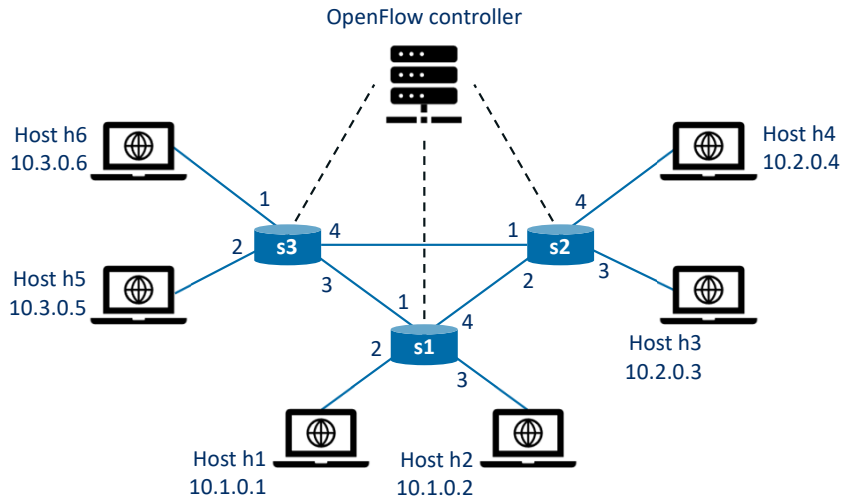
Generalized forwarding in OpenFlow (the most used SDN protocol)

- **Flow table** consists of 3 columns:
 - Header field values to match (including link layer, network layer and transport layer fields)
 - Counters keeping track of number of matches and time table entry was last updated
 - Actions to be taken (forwarding, load balancing, rewriting header fields, dropping packets, etc.)
- Matching is based on 11 packet-header fields and the incoming port ID



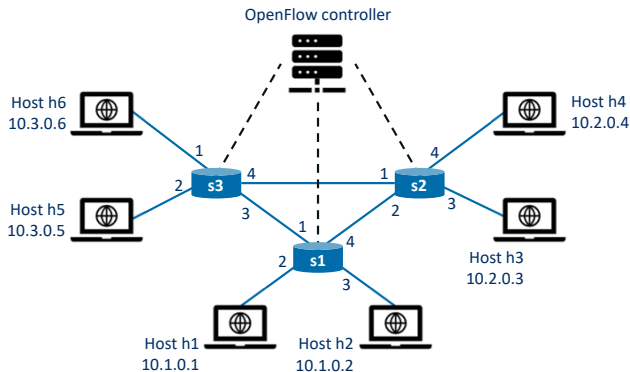
- Header values can have wildcards (e.g., 128.119.*.* matches all IP addresses starting with 128.119)
- Each flow table entry has a priority, used when multiple entries match with a single packet header

OpenFlow example network topology



Example 1: Simple forwarding

- Datagrams from h5 or h6 towards h3 or h4 should be forwarded via s1



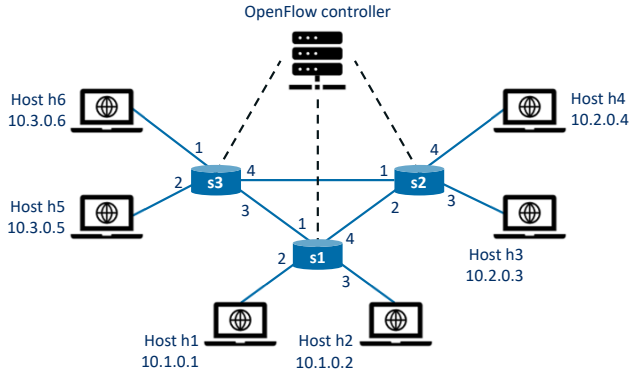
s1 flow table	
Match	Action
Ingress Port 1; IP Src = 10.3.0.*; IP Dst = 10.2.0.*	Forward(4)
...	...

s3 flow table	
Match	Action
IP Src = 10.3.0.*; IP Dst = 10.2.0.*	Forward(3)
...	...

s2 flow table	
Match	Action
Ingress port = 2; IP Dst = 10.2.0.3	Forward(3)
Ingress port = 2; IP Dst = 10.2.0.4	Forward(4)
...	...

Example 2: Load balancing

- Datagrams from h3 to 10.1.0.* should pass to s1
- Datagrams from h4 to 10.1.0.* should pass to s3



s2 flow table	
Match	Action
Ingress Port 3; IP Dst = 10.1.0.*	Forward(2)
Ingress Port 4; IP Dst = 10.1.0.*	Forward(1)
...	...

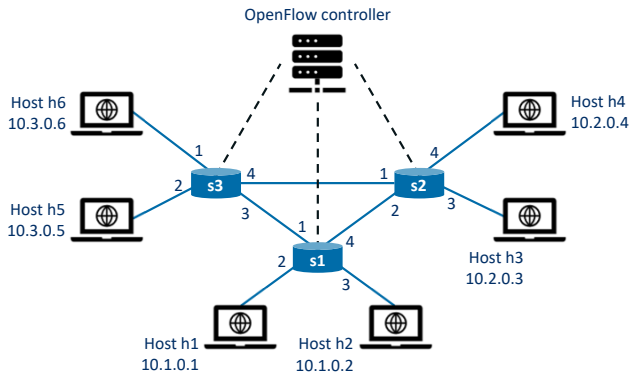


Homework: Write out the flow table entries for s1 and s3

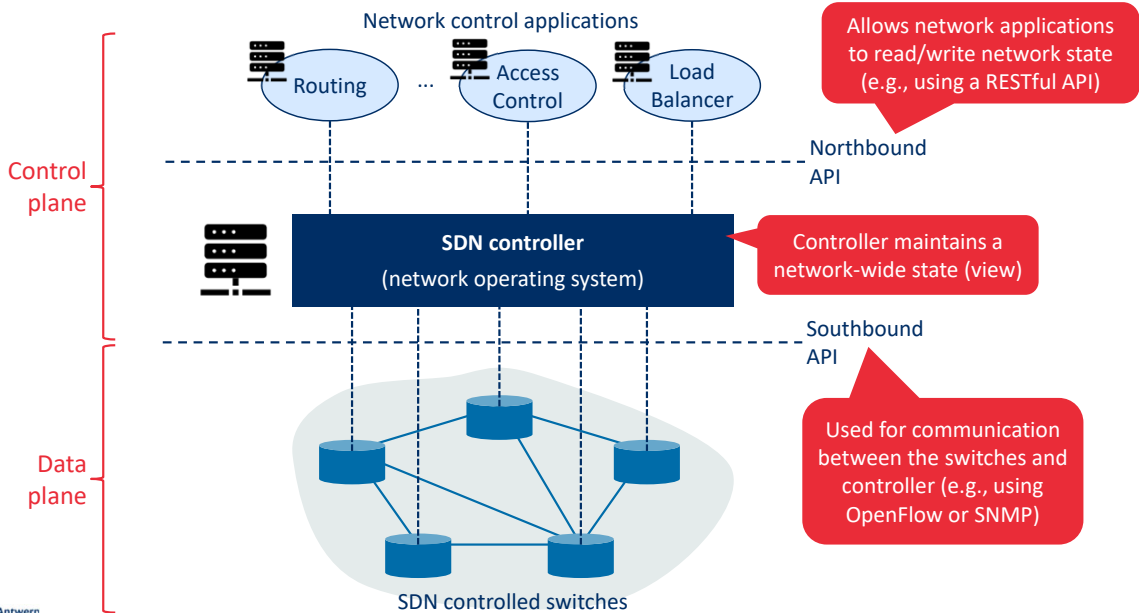
Example 3: Firewalling

- s2 only wants to receive traffic from hosts attached to s3

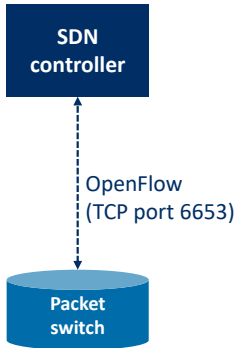
s2 flow table	
Match	Action
IP Src = 10.3.0.*; IP Dst = 10.2.0.3	Forward(3)
IP Src = 10.3.0.*; IP Dst = 10.2.0.4	Forward(4)



SDN architecture

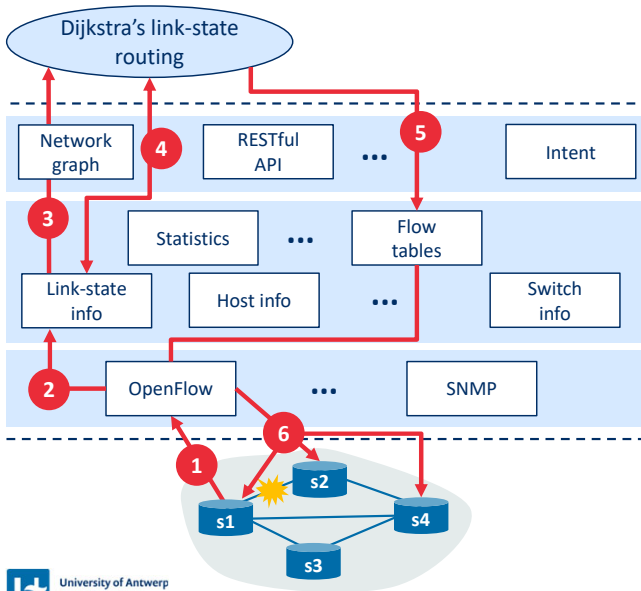


OpenFlow Protocol



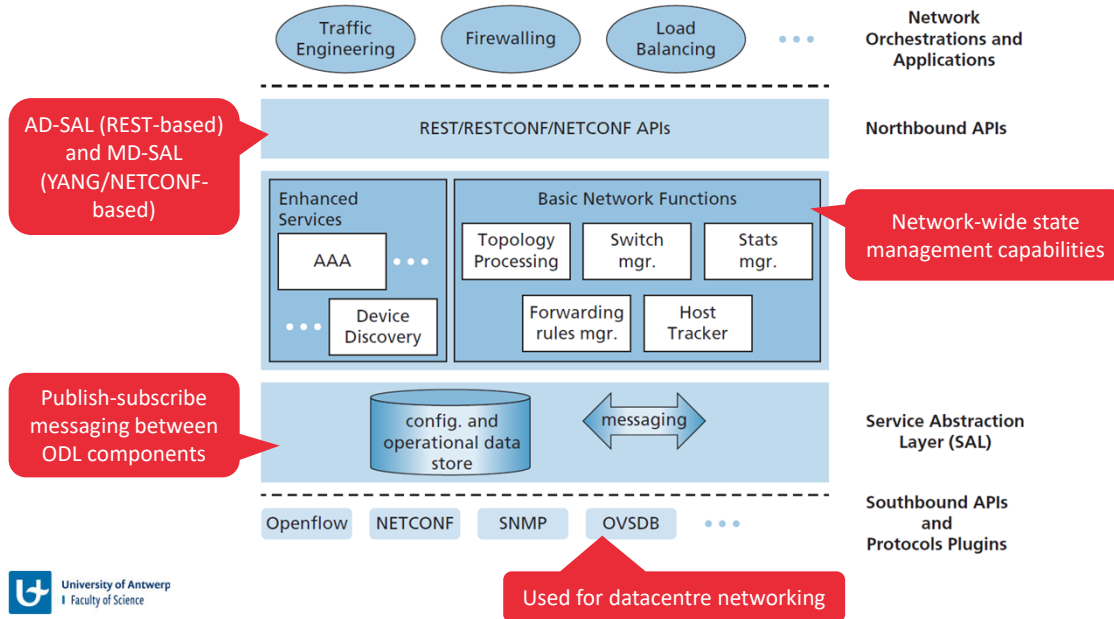
- Messages from **SDN controller** to **packet switch**
 - **Configuration**: Query and set the switch's configuration
 - **Modify-State**: Add/delete/modify entries in the switch's flow table and port properties
 - **Read-State**: Collect statistics and counter values from the switch's flow table and ports
 - **Send-Packet**: Send a specific packet via a specific port of the controlled switch
- Messages from **packet switch** to **SDN controller**
 - **Flow-Removed**: Sent when a flow table entry is removed (e.g., due to time-out or Modify-State)
 - **Port-Status**: Inform controller of a change in port status
 - **Packet-In**: Send a packet to the controller that does not match any flow table entry

Example: Dijkstra's shortest path routing using SDN



1. s1 detects link failure to s2 and notifies SDN controller using OpenFlow protocol
2. OpenFlow controller updates link-state database
3. Link-state routing application receives notification
4. Routing application calculates new routes based on updated link-state database
5. Routing application updates flow table database
6. OpenFlow is used to update flow tables in switches

SDN Controllers in practice: OpenDaylight



Network management protocols

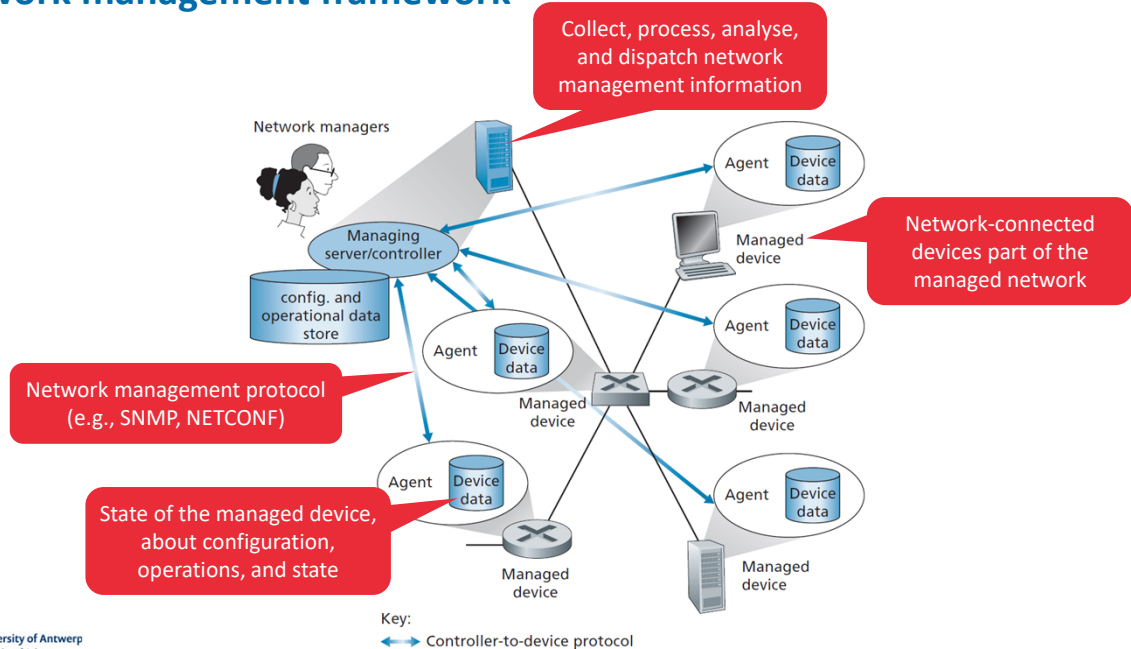
Internet Control Message Protocol (ICMP) – RFC 792

- Used by hosts and routers to communicate network-related information to each other
- ICMP messages are carried as IP payload (protocol number 1)
- Example messages:

Type	Code	Description
0	0	Echo reply (to ping)
3	0	Destination network unreachable
3	1	Destination host unreachable
3	2	Destination protocol unreachable
3	3	Destination port unreachable
3	6	Destination network unknown
3	7	Destination host unknown
8	0	Echo request
9	0	Router advertisement
10	0	Router discovery
11	0	TTL expired

- Ping makes use of ICMP (Type=8 and Type=0 messages)
- Traceroute captures TTL expired (Type=11, Code=0) and destination port unreachable (Type=3, Code=6) messages combined UDP datagrams with increasing TTL values

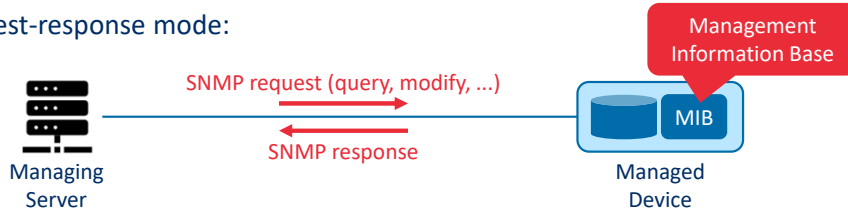
Network management framework



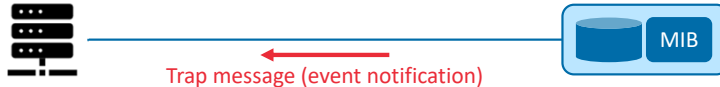
Simple Network Management Protocol (SNMP)

- Application-layer protocol (usually over UDP) used to convey network-management control information
- SNMPv3 is standardized in IETF RFC 3410

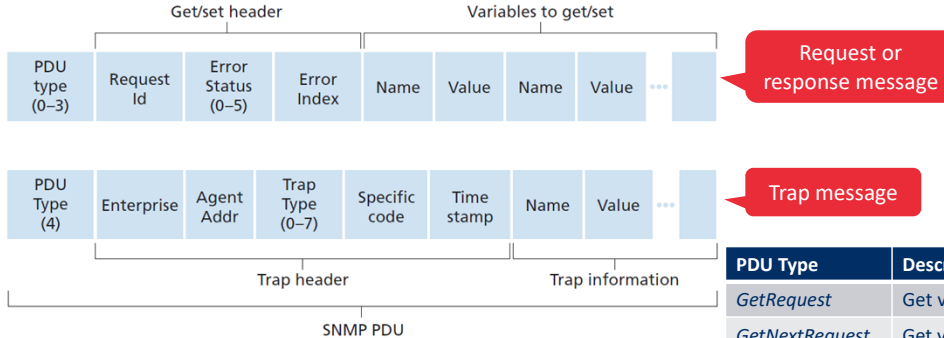
- Request-response mode:



- Trap-mode:



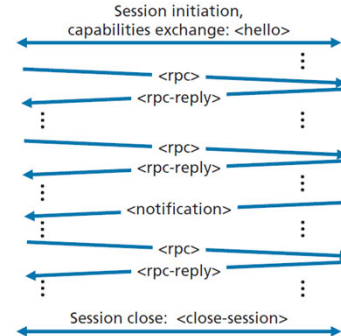
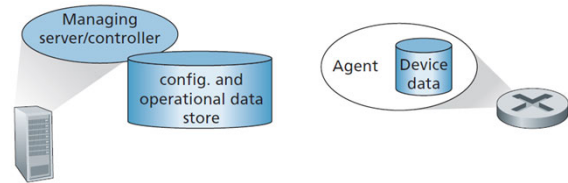
SNMP messages – aka. Protocol Data Units (PDUs)



PDU Type	Description
<i>GetRequest</i>	Get value of MIB object instance(s)
<i>GetNextRequest</i>	Get value of next MIB instance in list
<i>GetBulkRequest</i>	Get values of large block of MIB data
<i>InformRequest</i>	Inform remote managing entity of MIB values
<i>SetRequest</i>	Set value of MIB object instance(s)
<i>Response</i>	Response to PDU request messages
<i>Trap</i>	Device informs manager of exceptional event

Network Configuration Protocol (NETCONF) – RFC 6241

- Messaging protocol between managing server and managed network devices, that allows:
 - Retrieving, setting, and modifying configuration data
 - Querying operational data and statistics
 - Subscribe to notifications generated by devices
- Configurations are structured in XML documents
- NETCONF uses remote procedure calls (RPCs)
- Runs on top of TLS/TCP
- Semantics of the data are captured using the YANG data modelling language



NETCONF operations

NETCONF operation	Description
<get-config>	Retrieve all or part of a configuration
<get>	Retrieve all or part of a configuration state and operational state data
<edit-config>	Change all or part of a configuration
<lock>, <unlock>	Lock or unlock the configuration of a device, to avoid race conditions
<create-subscription>	Initiate even notification subscription
<notification>	Event notification for subscribers

Summary



Summary

- The Internet's network layer is best-effort and provides **addressing** (using IPv4 or IPv6)
- Every router performs **forwarding** (data plane) based on a forwarding table populated by a **routing** algorithm (control plane)
- Router **queueing** can occur both at the input and output ports
 - Queuing policies: First-in-first-out (FIFO), priority queuing, weighted fair queuing (and round robin)
- **IPv4** uses 32-bit addresses, while **IPv6** uses 128-bit addresses
 - Each IP address has a network prefix and a host identifier (marked using the subnet mask /x notation)
 - **DHCP** automatically assigns IP addresses, and discovers the default gateway, subnet mask, and local DNS server
 - **NAT** is used to translate between public and private IP addresses
- **Routing algorithms** determine the path from the source to destination host in a network
 - Centralized (**link-state**) vs. distributed (**distance vector/path vector**) routing
 - The Internet splits routing in two levels: within and between autonomous systems (ASs)
 - Several algorithms exist for intra-AS routing, such as **OSPF** (based on the link-state method)
 - Inter-AS routing is done using **BGP**, which is based on distance-vectors
- **SDN** supports more fine-grained packet processing, using multi-header matching rules and flexible actions
- ~~▪ Network management protocols (**SNMP** or **NETCONF**) are used manage (configure, monitor) network devices~~
- The contents of this part is covered in the **book** in Sections 4.1—4.4 and 5.2—5.5, 5.7

End of Part 4